

Team12_김영기님 크롤링 데이터에 대한 ISSUE 및 DBA 권장사항

1) 크롤링이 들어오면 “정규화 계층”을 3단으로 나누는 게 실무 표준

A. Raw(원문 보관) 계층 — “증거/재현/디버깅”

- 크롤링한 HTML/JSON 원문을 그대로 저장
- 파싱 로직이 바뀌어도 재파싱 가능
- 장애 분석/법적 이슈/원천 사이트 변경 대응에 유리

B. Listing(원천 상품) 계층 — “소스별 상품 단위”

- 쿠팡/스마트스토어/자사몰 등 **source + external_id** 기준으로 한 건의 원천상품
- 가격/재고/배송/리뷰/랭킹 같은 “자주 변하는 값”은 별도 스냅샷 테이블로 분리(정규화)

C. Canonical(정제 상품) 계층 — “서비스가 추천하는 표준 상품”

- 지금 ERD의 **products** 가 여기에 해당
- 여러 source listing을 하나의 canonical product에 매핑(중복 제거, 통합)

핵심 포인트:

크롤링 시스템은 “원천(source)”이 바뀌고 “가격/재고”가 계속 변합니다.

그래서 **products** 한 테이블에 다 때려 넣으면 6개월 후에 무조건 운영이 지옥이 됩니다.

2) 꼭 추가되면 좋은 테이블(정규화 확장 포인트) 상세 분석

아래는 “현 **products** 중심 모델”에 크롤링을 붙일 때, 실무에서 거의 반드시 들어가는 구성입니다.

(1) 소스/채널 정의: **crawl_sources**

왜 필요한가

- 같은 상품도 소스마다 ID 체계가 다르고, 크롤링 정책(레이트리밋/헤더/파서/robots)이 다름

- “어느 사이트에서 왔는지”가 PK의 일부가 됨

정규화 포인트

- `source_code` (unique)로 표준화, listings는 source_id FK만 들고 있음
-

(2) 크롤링 작업/실행 추적: `crawl_jobs` , `crawl_requests`

왜 필요한가(DBA 실무)

- 크롤링은 100% 장애가 납니다(차단/타임아웃/파서 깨짐/구조 변경)
- “언제, 무엇을, 왜 실패했는지”를 DB에 남겨야 **재시도/복구**가 가능합니다.

정규화 포인트

- Job(배치 단위)과 Request(페이지 단위)를 분리
 - request는 raw 저장과 연결됨(1:N)
-

(3) 원문 저장: `raw_documents`

왜 필요한가

- 파싱 결과만 저장하면 “파서 버그”가 났을 때 재현이 안 됨
- 소스 구조 변경 시 과거 데이터를 다시 파싱해야 할 때가 많음

실무 포인트

- 원문은 용량이 매우 큼 → 파티셔닝/TTL(보관기간)/압축 정책 필요
 - DB에 넣을지, S3/스토리지에 넣고 DB에는 경로만 돌지 결정해야 함
 - **MVP**: DB에 JSON/텍스트 저장 OK
 - **운영 규모**: 원문은 오브젝트 스토리지 + DB에는 메타/해시/경로만 추천
-

(4) 원천 상품: `product_listings`

정규화 핵심

- `products` (정제)와 분리해야 함
- listing은 “원천 소스의 상품 1개”이며, **source_id + external_id**가 유일 키

왜 이게 중요?

- “같은 canonical product”가 쿠팡에도 있고 스마트스토어에도 있음
- listing을 분리하면:

- 가격/재고는 listing 단위로 추적
- canonical product는 추천/검색의 기준으로 안정적으로 유지

(5) 가격/재고/배송 등 변동값: `listing_price_snapshots`, `listing_availability_snapshots`

정규화 핵심

- listing row를 업데이트로 덮어쓰지 말고, **스냅샷(시간축) 테이블**로 분리
- 이렇게 해야 "가격변동", "재고변동", "추천 후 구매율" 분석이 가능합니다.

DBA 실무 포인트

- 스냅샷 테이블은 가장 빨리 커짐 → 파티셔닝/BRIN 인덱스/보관정책 필수

(6) 이미지/옵션/속성(규격) 테이블

크롤링 상품에는 대개:

- 이미지 여러 장(1:N)
 - 옵션(사이즈/색상/좌우/규격)(1:N)
 - 속성(재질/호환/사이즈/ISO9999 코드 후보 등)(1:N)
- 이 붙습니다.

정규화 전략

- 단순히 JSONB로도 가능하지만, "조회/필터"가 요구되면 분리 테이블이 맞습니다.
 - 예: "무릎보조기 + 사이즈 L + 좌측" 필터링이 필요해지면 JSONB는 고통

(7) 중복 제거/매핑: `product_listing_map`

정규화 핵심

- listing(원천) ↔ product(정제) 관계는 거의 항상 **N:1** (여러 listing이 하나의 canonical product로 합쳐짐)
- 그리고 매핑에는 **신뢰도/매칭근거/승인여부** 같은 운영 필드가 필요합니다.

실무 포인트

- 자동 매칭 결과를 사람이 승인하는 플로우가 있으면 `match_status` 가 필수
- 추천에서 "정제상품(product)"를 쓰되, 실제 구매 링크는 "listing"을 타게 하는 구조가 많음

3) DBA 실무 적용 포인트(현업에서 안 하면 사고나는 것들)

A. 유일키/업서트 전략 (중복 삽입 방지)

- `product_listings` 에 반드시:
 - `UNIQUE(source_id, external_id)`
- `listing_price_snapshots` 는:
 - `(listing_id, captured_at)` 또는 `(listing_id, captured_date, captured_hour)` 같은 중복 방지 키 고려

이걸 안 하면

- 같은 상품이 매 크롤링마다 새 listing으로 생성 → 추천/통계 전부 깨짐

B. "큰 테이블"의 파티셔닝/인덱싱

크롤링/스냅샷은 데이터가 폭발합니다.

- 파티셔닝 추천 대상
 - `raw_documents`
 - `listing_price_snapshots`
 - `crawl_requests`
- 인덱스 전략
 - 스냅샷: `(listing_id, captured_at DESC)`
 - 시간 조건이 많은 테이블: BRIN(`captured_at`)도 고려(대용량에서 효율적)

C. 보관 정책(데이터 수명주기)

- 원문(raw)은 보관기간을 짧게 가져가는 경우가 많습니다(예: 30~90일)
- 스냅샷도 "최근 1년은 촘촘히, 그 이전은 일 단위 집계만 남기기" 같은 정책이 실무적

D. 크롤링 실패/재시도 설계

DB에 최소한 아래가 있어야 운영됩니다.

- `status` : `queued/running/succeeded/failed`
- `error_code` , `error_message`

- `attempt_count` , `next_retry_at`
- `http_status` , `response_time_ms`

E. 추천/전환 모델과 연결할 “정답 데이터” 정리

현재 ERD에서 `recommendations` 에 구매완료/금액이 있고, `conversion_events` 도 있습니다.

크롤링까지 들어오면 더 혼재되기 쉬워서, 다음 중 하나로 통일하는 게 좋습니다.

- 안1(추천): 구매/전환은 이벤트 로그(`conversion_events`)만 정답
 - `recommendations.purchase_completed` 같은 컬럼은 캐시/파생으로만 사용
- 안2: `purchases` 테이블을 별도로 두고 정답화
 - 결제/구매가 “정형 데이터”로 중요해지면 `purchases`가 좋음

4) 적용용 예시 DDL (PostgreSQL)

아래는 “현 `products` (정제 상품)가 이미 있다”는 가정 하에, 크롤링 확장에 필요한 최소 세트를 제공합니다.

```
-- 1) 소스 정의
CREATE TABLE IFNOTEXISTS crawl_sources (
  id          UUIDPRIMARY KEYDEFAULT uuid_generate_v4(),
  source_code TEXTNOT NULLUNIQUE,-- 'coupang', 'smartstore', 'selfmal
  '
  display_name TEXTNOT NULL,
  base_url    TEXT,
  is_active   BOOLEANNOT NULLDEFAULTTRUE,
  created_at  TIMESTAMPTZNOT NULLDEFAULT now()
);

-- 2) 크롤링 배치(Job)
CREATE TABLE IFNOTEXISTS crawl_jobs (
  id          UUIDPRIMARY KEYDEFAULT uuid_generate_v4(),
  source_id   UUIDNOT NULLREFERENCES crawl_sources(id)ONDELETE R
  Estrict,
  job_type    TEXTNOT NULL,-- 'search', 'detail', 'price_refresh'
  status      TEXTNOT NULLDEFAULT'queued',
  started_at  TIMESTAMPTZ,
```

```

finished_at TIMESTAMPTZ,
total_targets INTEGER NOT NULL DEFAULT 0,
success_count INTEGER NOT NULL DEFAULT 0,
fail_count INTEGER NOT NULL DEFAULT 0,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 3) 요청 단위(페이지/상품 URL)
CREATE TABLE IF NOT EXISTS crawl_requests (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  job_id      UUID NOT NULL REFERENCES crawl_jobs(id) ON DELETE CASCADE,
  target_url  TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'queued', -- queued/running/succeeded/failed
  http_status INTEGER,
  attempt_count INTEGER NOT NULL DEFAULT 0,
  next_retry_at TIMESTAMPTZ,
  error_code  TEXT,
  error_message TEXT,
  fetched_at  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_crawl_requests_job_status
ON crawl_requests(job_id, status);

CREATE INDEX IF NOT EXISTS idx_crawl_requests_next_retry
ON crawl_requests(next_retry_at)
WHERE next_retry_at IS NOT NULL;

-- 4) 원문 저장(간단 버전: DB에 직접 저장)
CREATE TABLE IF NOT EXISTS raw_documents (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  request_id  UUID NOT NULL REFERENCES crawl_requests(id) ON DELETE CASCADE,
  content_type TEXT NOT NULL, -- 'text/html', 'application/json'
  content_text TEXT, -- HTML/JSON text

```

```

content_hash TEXT,-- dedupe/변경감지용
created_at TIMESTAMPTZNOT NULLDEFAULT now()
);

CREATE INDEX IFNOTEXISTS idx_raw_documents_request
ON raw_documents(request_id);

-- 5) 원천 상품(listing)
CREATE TABLE IFNOTEXISTS product_listings (
  id          UUIDPRIMARY KEYDEFAULT uuid_generate_v4(),
  source_id   UUIDNOT NULLREFERENCES crawl_sources(id)ONDELETE
RESTRICT,
  external_id TEXTNOT NULL,-- 소스 상품ID
  product_url TEXTNOT NULL,
  title       TEXT,
  brand       TEXT,
  seller_name TEXT,
  currency    TEXTNOT NULLDEFAULT'KRW',
  last_crawled_at TIMESTAMPTZ,
  is_activeBOOLEANNOT NULLDEFAULTTRUE,
  created_at  TIMESTAMPTZNOT NULLDEFAULT now(),
  updated_at  TIMESTAMPTZNOT NULLDEFAULT now(),
  UNIQUE (source_id, external_id)
);

CREATE INDEX IFNOTEXISTS idx_product_listings_source_active
ON product_listings(source_id, is_active);

-- 6) 가격 스냅샷(변동값)
CREATE TABLE IFNOTEXISTS listing_price_snapshots (
  id          UUIDPRIMARY KEYDEFAULT uuid_generate_v4(),
  listing_id  UUIDNOT NULLREFERENCES product_listings(id)ONDELETE C
ASCADE,
  priceNUMERIC(12,2),
  shipping_feeNUMERIC(12,2),
  stock_status TEXT,-- 'in_stock','out_of_stock','unknown'
  captured_at TIMESTAMPTZNOT NULLDEFAULT now()
);

```

```
CREATE INDEX IFNOTEXISTS idx_listing_price_listing_time
ON listing_price_snapshots(listing_id, captured_atDESC);

-- 7) 정제 상품(products)과 매핑 (N listings → 1 product)
CREATE TABLE IFNOTEXISTS product_listing_map (
  listing_id    UUIDPRIMARY KEYREFERENCES product_listings(id)ONDELETE CASCADE,
  product_id    UUIDNOT NULLREFERENCES products(id)ONDELETE RESTRICT,
  match_status  TEXTNOT NULLDEFAULT'auto',-- auto/manual/rejected
  match_score   NUMERIC(5,2),
  match_reason  TEXT,
  matched_at    TIMESTAMPTZNOT NULLDEFAULT now()
);

CREATE INDEX IFNOTEXISTS idx_product_listing_map_product
ON product_listing_map(product_id);
```

이 구조만 있어도 “원천 상품 수집 → 가격 변동 추적 → 정제 상품으로 추천”이 가능해집니다.

이후 필요해지면 `listing_images`, `listing_attributes`, `listing_options` 를 추가로 정규화하면 됩니다.

5) 운영에서 바로 체감되는 “DBA 체크리스트” (현장 포인트)

1) 데이터 폭증 테이블부터 파티션/보관정책을 먼저 정해라

- `listing_price_snapshots`, `raw_documents`, `crawl_requests`
- “무한정 쌓아두기”는 100% 장애로 돌아옵니다(백업, vacuum, 인덱스, 비용).

2) 중복 방지 키(UNIQUE) 없으면 크롤링은 망한다

- listing은 `source_id + external_id` 유니크가 사실상 생명줄
- price snapshot은 중복 방지 정책을 별도로 설계(동일 시각 중복 삽입 방지)

3) 원문 저장은 “DB vs 스토리지”를 조기에 결정

- DB 저장: 개발/디버깅 쉬움, 비용/용량 위험
- 스토리지 저장: 운영에 유리, 다만 조회/권한/정합성 설계 필요
실무에서는 원문은 스토리지 + DB에 해시/경로만이 오래갑니다.

4) 추천/구매/전환 “정답 테이블”을 하나로 고정

- `recommendations` 에 구매여부를 두면 편하지만, 이벤트 로그와 충돌하기 쉬움
- 정답은 `conversion_events` (또는 `purchases`)로 고정하고 나머지는 파생값으로 운용 추천

5) 장애/차단/재시도는 애초에 데이터 모델에 넣어라

- `attempt_count` , `next_retry_at` , `error_*` 없이 운영하면
“왜 누락됐지?”를 영원히 못 잡습니다.

아래는 현재 ERD의 `products / recommendations / conversion_events` 를 “크롤링(원천 listing)”까지 포함해 운영 가능한 구조로 확장하는 실무 설계안입니다.

(참고: 이전에 올려주신 일부 파일이 만료된 상태라, 지금은 ERD에서 보이는 테이블명 기준 + 일반적으로 충돌 없는 확장 DDL로 작성했습니다. `DDL.sql` 을 기준으로 정확히 FK/컬럼명을 1:1로 맞춘 “완전 통합 스크립트”가 필요하다면 파일을 다시 올려주시면 그 버전으로 재정렬 해드릴 수 있어요.)

1) 연결 방식 대안 3가지 비교

(추천 링크를 `listing` 으로 쓰는지 / `product` 로 쓰는지)

공통 전제: “정제상품 vs 원천상품”

- `products` = **canonical(정제) 상품**: 서비스가 보여주는 표준 상품(중복 제거, 설명/이미지/카테고리 등)
- `product_listings` = **source listing(원천) 상품**: 쿠팡/스마트스토어/자사몰 등 소스별 상품 단위 (`source_id` + `external_id`로 유니크)

대안 A) 추천은 “product 기준”, 클릭/구매는 “listing 선택” (권장, 가장 균형형)

구조

- `recommendations` : (consultation_id, **product_id**)
- `conversion_events` : (recommendation_id, **listing_id** nullable/필수, event_type, amount, ...)
- `product_listing_map` : listing → product 매핑

장점

- UI/추천 품질: 추천/검색은 **product** 중심으로 깔끔
- 원천 링크/가격은 listing에서 가져오면 되므로 **가격 변동/재고 변동에 유리**
- “같은 제품을 여러 마켓에서 판매”를 자연스럽게 처리

단점/운영 포인트

- 클릭 시점에 “어느 listing을 보여줄지” 결정 로직 필요
(예: 최저가/신뢰도 높은 소스/재고 있는 listing 우선)

추천 사용 케이스

- 추천 품질(제품 단위)이 핵심이고, **실구매 링크는 여러 소스 중 선택해야 하는 서비스**

대안 B) 추천 자체를 “listing 기준”으로 저장 (단순, 실구매 트래킹은 쉬움)

구조

- `recommendations` : (consultation_id, **listing_id**, product_id optional)
- `conversion_events` : recommendation_id 중심

장점

- 추천 → 클릭 → 구매가 **항상 동일 URL**로 이어져 트래킹이 단순
- “어떤 소스의 어떤 상품이 성과가 좋았는지” 분석이 쉽다

단점/운영 포인트

- 동일 제품이 소스마다 존재하면 **추천이 중복으로 보이는 UX**가 생김
- “제품 단위 분석”을 하려면 결국 product 매핑이 필요

추천 사용 케이스

- 1~2개 소스만 쓰고, **마켓별 차이가 큰 서비스**(가격/배송/옵션이 핵심)

대안 C) 하이브리드: 추천은 product, “추천 당시 대표 listing”을 함께 고정

구조

- **recommendations** : (consultation_id, product_id, **primary_listing_id**)
- 이벤트는 listing_id를 그대로 쓰거나, primary_listing_id를 기본값으로 사용

장점

- 추천 화면에서 링크가 즉시 결정(단순 UX)
- 추천 당시의 “대표 소스/대표 가격” 스냅샷에 유리

단점/운영 포인트

- primary_listing이 품질/차단/링크 변경 시 **대체 listing 선택 로직**이 필요
- 추천 당시 가격과 현재 가격 차이 표시 정책 필요(법/신뢰)

추천 사용 케이스

- 추천 결과에 “바로가기 링크”가 중요하고, “대표 소스” 운영 정책이 있는 서비스

DBA 관점 결론(추천)

- 운영 안정성과 분석 밸런스가 가장 좋은 건 **대안 A**
- 소스가 적고 트래킹 단순화가 최우선이면 **대안 B**
- UX 단순 + 운영정책(대표마켓)이 있으면 **대안 C**

2) 확장 테이블 포함 최종 DDL + 인덱스/파티션/보관정책 템플릿 (PostgreSQL)

아래 DDL은 기존 `products`, `recommendations`, `conversion_events` 가 이미 존재한다고 가정하고 "크롤링 확장"만 추가합니다.

UUID는 `pgcrypto` 의 `gen_random_uuid()` 를 사용합니다.

pg_cron은 환경에 따라 미설치일 수 있으니, 보관정책 실행은 "DB 함수 + 외부 스케줄러(서버 cron/윈도우 작업 스케줄러)" 형태를 권장합니다.

2-1) 크롤링/원천/스냅샷 테이블 DDL

```
BEGIN;

-- UUID 생성
CREATE EXTENSION IFNOTEXISTS pgcrypto;

-- 1) 소스(마켓/채널) 정의
CREATE TABLE IFNOTEXISTS crawl_sources (
  id          uuidPRIMARY KEYDEFAULT gen_random_uuid(),
  source_code textNOT NULLUNIQUE,-- 'coupang','smartstore','selfmall'
  display_name textNOT NULL,
  base_url    text,
  is_activebooleanNOT NULLDEFAULTtrue,
  created_at  timestamptzNOT NULLDEFAULT now()
);

-- 2) 크롤링 Job(배치 단위)
CREATE TABLE IFNOTEXISTS crawl_jobs (
  id          uuidPRIMARY KEYDEFAULT gen_random_uuid(),
  source_id   uuidNOT NULLREFERENCES crawl_sources(id)ONDELETE RE
STRICT,
  job_type    textNOT NULL,-- 'search','detail','price_refresh'
  status      textNOT NULLDEFAULT'queued',-- queued/running/succeede
d/failed
  started_at  timestamptz,
  finished_at timestamptz,
  total_targetsintegerNOT NULLDEFAULT0,
  success_countintegerNOT NULLDEFAULT0,
  fail_countintegerNOT NULLDEFAULT0,
```

```

    metadata    jsonb,
    created_at  timestampz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_crawl_jobs_source_status
ON crawl_jobs(source_id, status, created_at DESC);

-- 3) 크롤링 Request(페이지 단위) - 파티션 대상(대용량)
CREATE TABLE IF NOT EXISTS crawl_requests (
    id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id      uuid NOT NULL REFERENCES crawl_jobs(id) ON DELETE CASCADE,
    target_url   text NOT NULL,
    status      text NOT NULL DEFAULT 'queued', -- queued/running/succeeded/failed
    http_status integer,
    response_time_ms integer,
    attempt_count integer NOT NULL DEFAULT 0,
    next_retry_at timestampz,
    error_code   text,
    error_message text,
    fetched_at   timestampz,
    created_at   timestampz NOT NULL DEFAULT now()
) PARTITION BY RANGE (created_at);

-- (예시) 월 파티션 2개 생성 (운영에서는 자동 생성 함수 사용 권장)
CREATE TABLE IF NOT EXISTS crawl_requests_2025_01
PARTITION OF crawl_requests FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE IF NOT EXISTS crawl_requests_2025_02
PARTITION OF crawl_requests FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');

CREATE INDEX IF NOT EXISTS idx_crawl_requests_job_status
ON crawl_requests (job_id, status);

CREATE INDEX IF NOT EXISTS idx_crawl_requests_next_retry
ON crawl_requests (next_retry_at)

```

```
WHERE next_retry_at IS NOT NULL;
```

-- 4) 원문(Raw) 저장 - 파티션 대상(대용량)

-- 운영 규모가 커지면 content_text는 오브젝트 스토리지로 빼고 storage_key만 두는 방식을 추천

```
CREATE TABLE IF NOT EXISTS raw_documents (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  request_id  uuid NOT NULL REFERENCES crawl_requests(id) ON DELETE CASCADE,  
  content_type text NOT NULL, -- 'text/html', 'application/json'  
  content_text text, -- or NULL if stored externally  
  storage_key text, -- e.g. 's3://bucket/key' (선택)  
  content_hash text, -- 변경감지/dedupe  
  created_at  timestampz NOT NULL DEFAULT now()  
) PARTITION BY RANGE (created_at);
```

```
CREATE TABLE IF NOT EXISTS raw_documents_2025_01  
PARTITION OF raw_documents FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
```

```
CREATE TABLE IF NOT EXISTS raw_documents_2025_02  
PARTITION OF raw_documents FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
```

```
CREATE INDEX IF NOT EXISTS idx_raw_documents_request  
ON raw_documents (request_id);
```

```
CREATE INDEX IF NOT EXISTS idx_raw_documents_hash  
ON raw_documents (content_hash);
```

-- 5) 원천 상품 Listing(소스별 상품)

```
CREATE TABLE IF NOT EXISTS product_listings (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  source_id   uuid NOT NULL REFERENCES crawl_sources(id) ON DELETE RESTRICT,  
  external_id text NOT NULL, -- 소스 상품 ID  
  product_url text NOT NULL,  
  title       text,  
  brand       text,
```

```

seller_name    text,
currency       textNOT NULLDEFAULT'KRW',
is_active      booleanNOT NULLDEFAULTtrue,
last_crawled_at timestampz,
created_at     timestampzNOT NULLDEFAULT now(),
updated_at     timestampzNOT NULLDEFAULT now(),
UNIQUE (source_id, external_id)
);

CREATE INDEX IFNOTEXISTS idx_product_listings_source_active
ON product_listings (source_id, is_active);

CREATE INDEX IFNOTEXISTS idx_product_listings_updated
ON product_listings (updated_atDESC);

-- 6) Listing 변동 스냅샷(가격/재고) - 파티션 대상(초대용량)
CREATE TABLE IFNOTEXISTS listing_price_snapshots (
  id          uuidPRIMARY KEYDEFAULT gen_random_uuid(),
  listing_id   uuidNOT NULLREFERENCES product_listings(id)ONDELETE CASCADE,
  price        numeric(12,2),
  shipping_fee numeric(12,2),
  stock_status text,-- 'in_stock','out_of_stock','unknown'
  captured_at  timestampzNOT NULLDEFAULT now()
)PARTITIONBYRANGE (captured_at);

CREATE TABLE IFNOTEXISTS listing_price_snapshots_2025_01
PARTITIONOF listing_price_snapshotsFORVALUESFROM ('2025-01-01')TO
('2025-02-01');
CREATE TABLE IFNOTEXISTS listing_price_snapshots_2025_02
PARTITIONOF listing_price_snapshotsFORVALUESFROM ('2025-02-01')TO
('2025-03-01');

CREATE INDEX IFNOTEXISTS idx_listing_price_listing_time
ON listing_price_snapshots (listing_id, captured_atDESC);

-- 7) 정제상품(products) ↔ listing 매핑 (중복 제거/통합의 핵심)
CREATE TABLE IFNOTEXISTS product_listing_map (

```

```

listing_id    uuidPRIMARY KEYREFERENCES product_listings(id)ONDELET
E CASCADE,
product_id    uuidNOT NULLREFERENCES products(id)ONDELETE RESTR
ICT,
match_status  textNOT NULLDEFAULT'auto',-- auto/manual/rejected
match_scorenumeric(5,2),
match_reason  text,
matched_by    uuid,-- users.id (검수자)
matched_at    timestamptzNOT NULLDEFAULT now()
);

CREATE INDEX IFNOTEXISTS idx_product_listing_map_product
ON product_listing_map (product_id);

COMMIT;

```

2-2) **recommendations / conversion_events** 에 listing 연결을 넣는 DDL (대안 A/C 공통)

대안 A를 기준으로 “클릭/구매 이벤트가 어떤 listing에서 발생했는지”를 남기려면 **conversion_events** 에 listing_id를 추가하는 게 가장 깔끔합니다.

```

-- (대안 A/C 권장) conversion_events에 listing_id 추가
ALTER TABLE conversion_events
ADDCOLUMN IFNOTEXISTS listing_id uuidREFERENCES product_listings(i
d)ONDELETESETNULL;

CREATE INDEX IFNOTEXISTS idx_conversion_events_listing_time
ON conversion_events (listing_id, created_atDESC);

```

대안 C를 쓰면 recommendations에 대표 listing도 추가:

```

-- (대안 C) recommendations에 대표 listing을 고정
ALTER TABLE recommendations
ADDCOLUMN IFNOTEXISTS primary_listing_id uuidREFERENCES product_li
stings(id)ONDELETESETNULL;

```

```
CREATE INDEX IFNOTEXISTS idx_recommendations_primary_listing
ON recommendations(primary_listing_id);
```

2-3) 파티션 자동 생성/보관정책(템플릿)

(1) 월 파티션 생성 함수 템플릿

┃ 운영에서는 “매월 1일”에 다음 달 파티션까지 미리 생성하는 방식 추천

```
CREATE OR REPLACE FUNCTION ensure_monthly_partition(
    p_parent regclass,
    p_col_name text,
    p_monthdate
) RETURNS void AS $$
DECLARE
    v_startdate := date_trunc('month', p_monthdate)::date;
    v_enddate := (date_trunc('month', p_monthdate) + interval '1 month')::date;
    v_tbl text := format('%s_%s', p_parent::text, to_char(v_startdate, 'YYYY_MM'));
    v_sql text;
BEGIN
    v_sql := format(
        'CREATE TABLE IF NOT EXISTS %I PARTITION OF %s FOR VALUES FROM
        (%L) TO (%L);',
        v_tbl, p_parent, v_startdate::text, v_enddate::text
    );
    EXECUTE v_sql;
END;
$$ LANGUAGE plpgsql;
```

사용 예:

```
SELECT ensure_monthly_partition('crawl_requests', 'created_at', current_date);
SELECT ensure_monthly_partition('raw_documents', 'created_at', current_date);
```

```
e);
SELECT ensure_monthly_partition('listing_price_snapshots','captured_at',current_date);
```

(2) 보관정책(오래된 파티션 삭제) 템플릿

keep_months 기준으로 이전 파티션 DROP (raw는 1~3개월, price snapshot은 6~12개월 등 정책 분리 권장)

```
CREATE OR REPLACE FUNCTION drop_partitions_older_than(
    p_parent regclass,
    p_keep_months int
) RETURNS void AS $$
DECLARE
    r record;
    v_cutoffdate := (date_trunc('month', now()) - (p_keep_months || ' months')::interval)::date;
BEGIN
    FOR r IN
        SELECT c.relname AS child
        FROM pg_inherits
        JOIN pg_class c ON pg_inherits.inhrelid = c.oid
        JOIN pg_class p ON pg_inherits.inhparent = p.oid
        WHERE p.oid = p_parent
    LOOP
        -- 테이블명 끝 YYYY_MM 파싱 가정 (예: listing_price_snapshots_2025_01)
        -- 실무에서는 파티션 범위를 pg_get_expr로 읽어 더 안전하게 처리 가능
        IF substring(r.child from '\d{4}_\d{2}$') IS NOT NULL THEN
            IF to_date(substring(r.child from '\d{4}_\d{2}$'), 'YYYY-MM') < v_cutoffdate THEN
                EXECUTE format('DROP TABLE IF EXISTS %I;', r.child);
            END IF;
        END IF;
    END LOOP;
END;
```

```
$$LANGUAGE plpgsql;
```

실행 예(외부 스케줄러에서):

```
SELECT drop_partitions_older_than('raw_documents',3);  
SELECT drop_partitions_older_than('crawl_requests',6);  
SELECT drop_partitions_older_than('listing_price_snapshots',12);
```

3) “중복 상품 매칭(자동/수동 승인)” 운영 테이블 설계

(match_queue / match_rules / 로그 포함)

3-1) 핵심 개념

- 자동 매칭은 100% 완벽하지 않습니다.
- 운영에서 필요한 건:
 1. 자동 후보 생성(rule 기반 + score)
 2. 검수 큐(pending → approved/rejected)
 3. 승인 시 **product_listing_map** 갱신
 4. “왜 매칭됐는지/누가 승인했는지” 감사 추적

3-2) DDL: match_rules / match_queue / match_audit_logs

```
BEGIN;  
  
CREATE EXTENSION IFNOTEXISTS pgcrypto;  
  
-- 1) 매칭 룰(자동 매칭 로직의 설정)  
CREATE TABLE IFNOTEXISTS match_rules (  
  id          uuidPRIMARY KEYDEFAULT gen_random_uuid(),
```

```

rule_name textNOT NULLUNIQUE,
is_enabledbooleanNOT NULLDEFAULTtrue,

-- 룰 타입 예시:
-- 'exact_external_id', 'url_normalize', 'title_similarity', 'brand_model', 'embedding'
rule_type textNOT NULL,

-- 룰별 설정(임계값, 가중치, 필드 매핑 등)
config jsonbNOT NULLDEFAULT'{}'::jsonb,

priorityintNOT NULLDEFAULT100,-- 낮을수록 먼저 적용
created_at timestamptzNOT NULLDEFAULT now(),
updated_at timestamptzNOT NULLDEFAULT now()
);

CREATE INDEX IFNOTEXISTS idx_match_rules_enabled_priority
ON match_rules(is_enabled, priority);

-- 2) 매칭 큐(검수 대상)
-- listing 1건에 대해 여러 후보(product 후보)가 들어갈 수 있음
CREATE TABLE IFNOTEXISTS match_queue (
  id uuidPRIMARY KEYDEFAULT gen_random_uuid(),
  listing_id uuidNOT NULLREFERENCES product_listings(id)ONDELETE CASCADE,
  candidate_product_id uuidNOT NULLREFERENCES products(id)ONDELETE RESTRICT,

  rule_id uuidREFERENCES match_rules(id)ONDELETESETNULL,
  match_scorenumeric(6,3)NOT NULLDEFAULT0,
  match_reason text,
  evidence jsonb,-- 어떤 필드가 일치했는지, 유사도 값 등

  status textNOT NULLDEFAULT'pending',-- pending/approved/rejected/expired
  reviewed_by uuid,-- users.id
  reviewed_at timestamptz,

```

```

created_at      timestampzNOT NULLDEFAULT now(),

-- 같은 listing에 같은 후보가 중복 생성되는 것을 방지
UNIQUE(listing_id, candidate_product_id)
);

CREATE INDEX IFNOTEXISTS idx_match_queue_status_score
ON match_queue(status, match_scoreDESC, created_atDESC);

CREATE INDEX IFNOTEXISTS idx_match_queue_listing
ON match_queue(listing_id);

-- 3) 매칭 감사 로그(누가 무엇을 승인/거절했는지)
CREATE TABLE IFNOTEXISTS match_audit_logs (
  id          uuidPRIMARY KEYDEFAULT gen_random_uuid(),
  listing_id  uuidNOT NULLREFERENCES product_listings(id)ONDELETE CASCADE,
  product_id  uuidREFERENCES products(id)ONDELETESETNULL,
  action      textNOT NULL,-- 'approve','reject','auto_map','unmap'
  actor_id    uuid,-- users.id or NULL for system
  detail      jsonb,
  created_at  timestampzNOT NULLDEFAULT now()
);

CREATE INDEX IFNOTEXISTS idx_match_audit_listing_time
ON match_audit_logs(listing_id, created_atDESC);

COMMIT;

```

3-3) 운영 플로우(실무 포인트)

자동 매칭 파이프라인 (추천)

1. 크롤링 → `product_listings` upsert
2. 후보 생성:
 - 를 실행해서 후보(product)를 찾고 `match_queue` 에 `pending` 으로 삽입

3. 임계치 이상은 자동 승인:

- $\text{score} \geq 0.95$ 같은 경우 `product_listing_map` 에 바로 반영하고 로그 남김

4. 나머지는 검수 UI에서 승인/거절:

- 승인 시: `product_listing_map(listing_id → product_id)` upsert
- 거절 시: `match_queue.status='rejected'`

5. 재크롤링/상품 변경 감지 시:

- listing의 `content_hash` /title/brand 변화가 크면 `match_queue` 를 재오픈하거나 `expired` 처리

DBA가 반드시 챙길 것

• 중복 방지 키

- `product_listings UNIQUE(source_id, external_id)`
- `match_queue UNIQUE(listing_id, candidate_product_id)`

• 상태 기반 인덱스

- `match_queue(status, score DESC)`

• 감사 로그(누가 승인했나)

- 운영에서 분쟁/품질 이슈 생기면 이게 생명줄

• 자동 매칭과 수동 매칭이 충돌하지 않도록

- `product_listing_map.match_status` 를 `auto/manual` 로 분리하고, 수동 승인 시 manual이 auto를 덮어쓰게 정책화

마지막 정리: 추천 조합

- 연결 방식: 대안 A (product 추천 + listing 이벤트 기록)
- 크롤링 확장: `crawl_*` + `product_listings` + `listing_price_snapshots` + `product_listing_map`
- 매칭 운영: `match_rules` + `match_queue` + `match_audit_logs`
- 보관정책: raw는 1~3개월, request는 3~6개월, price snapshot은 6~12개월(서비스 성장에 맞춰 조정)